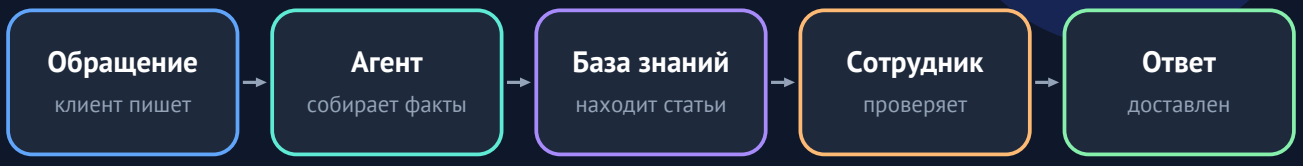
УЧЕБНЫЙ ПРОЕКТ · ВЫМЫШЛЕННАЯ КОМПАНИЯ TECHFLOW

AI Support Agent

Наглядный гайд по проекту: как ИИ-помощник готовит ответ оператору поддержки - от обращения клиента до одобренного человеком ответа.



Решение всегда за человеком: агент только готовит черновик.

7 / 10	635	22 / 27	5	4
этапов дорожной карты готово	автоматических тестов	пути / операции HTTP API	инструментов агента	миграции базы данных

Что внутри

1 Проект за минуту
2 Архитектура
3 Путь обращения
4 Как работает агент
5 База знаний (RAG)
6 LangGraph и решение человека
7 Данные и миграции
8 Шпаргалка API
9 Как запустить
10 Проверки и CI/CD
11 Дорожная карта
12 Словарь
13 Итоги аудита
14 Матрица CI
15 Защита и доставка
16 Полный прогон
17 Путь оператора
18 Перезапуск
19 Диагностика
20 Следующие шаги

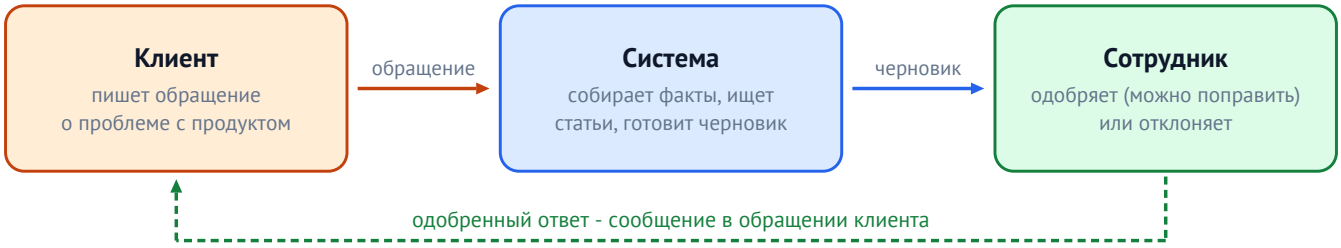
Аудит 2026-10-18 · основа f6e72ae · mock без платных запросов

Подробные пошаговые инструкции - в docs/STAGE_01-07.md и docs/instructions/

1 Проект за одну минуту

Зачем нужен помощник оператора и что он делает - без технических деталей.

Оператор технической поддержки тратит много времени на сбор фактов: кто клиент, действует ли его лицензия, обращался ли он раньше, что сказано в документации. **AI Support Agent** собирает эти факты автоматически и готовит черновик ответа со ссылками на статьи базы знаний. **Отправлять ли ответ, решает человек.**



Что делает

Читает обращение, клиента, лицензию и историю; ищет статьи; готовит черновик со ссылками; ждёт решения сотрудника; доставляет одобренный ответ сообщением в обращении.

Чего не делает

Не отправляет письма; не меняет обращение без человека; не вызывает платную модель по умолчанию (режим mock); пока нет входа сотрудников и прав доступа.

Главные принципы

Человек в контуре; агент только читает данные; безопасные ошибки без секретов; каждое изменение проверяют тесты и CI перед публикацией образа.

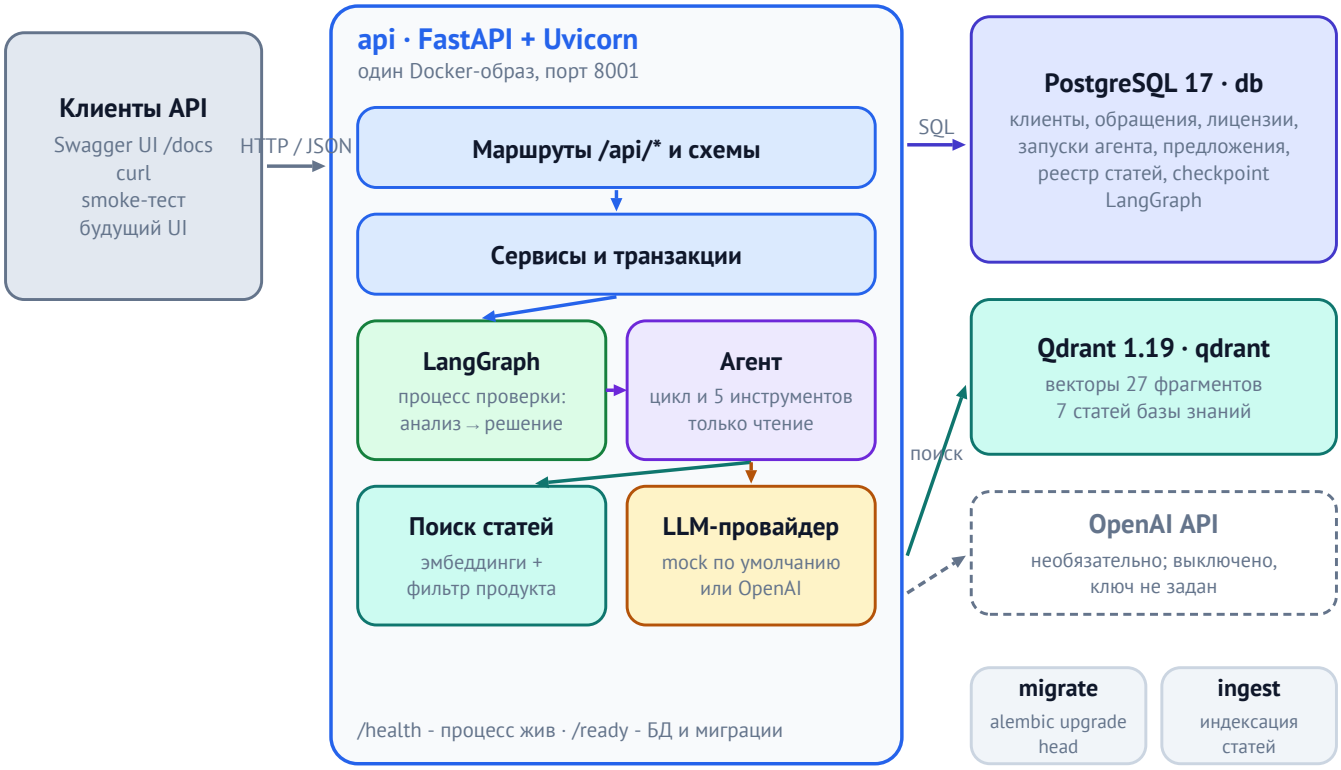
Как устроен проект

Слой	Технология	Зачем
HTTP API	FastAPI + Uvicorn	Маршруты, проверка JSON, документация Swagger
Данные	PostgreSQL 17, SQLAlchemy 2, Alembic	Записи, транзакции, версии схемы
Агент	Свой цикл tool calling	5 инструментов только для чтения, лимиты, проверка итога
Поиск	Qdrant 1.19 + эмбеддинги	Статьи базы знаний по смыслу (mock - по словам)
Процесс	LangGraph + PostgresSaver	Ожидание решения человека, переживает перезапуск
Доставка	Docker Compose, GitHub Actions, GHCR	Запуск одной командой и проверенный образ

Важно: TechFlow, клиенты и статьи - вымышленные. Режим **mock** по умолчанию работает без интернета и ключей: он показывает путь данных, но не качество настоящей модели. Подключение OpenAI подготовлено, но в проекте ещё не включалось.

2 Архитектура

Из каких частей состоит система и как они связаны.



Сервисы Docker Compose

Сервис	Что это	Порт на Mac	Данные
`db`	PostgreSQL 17	5433	том postgres_data
`qdrant`	Qdrant 1.19.1, телеметрия выключена	6335	том qdrant_data
`migrate`	разовый: alembic upgrade head	-	-
`api`	FastAPI; стартует после migrate и qdrant	8001	-
`db-test`	тестовая PostgreSQL (профиль test)	5434	в памяти
`qdrant-test`	тестовый Qdrant (профиль test)	6336	в памяти

Порядок запуска: **db healthy** → **migrate завершился** → **qdrant healthy** → **api healthy**. Обычный `docker compose down` сохраняет тома: записи, индекс статей и ожидающие решения переживают пересоздание контейнеров. `down --volumes` удаляет всё.

3 Путь обращения: от создания до ответа

Восемь шагов, через которые проходит каждое обращение. Справа - где это видно в API.

1. **Создать данные**
   Клиент, продукт, лицензия и обращение (статус open).
   `POST /api/tickets`

2. **Запустить анализ**
   Стартует граф LangGraph; первый узел - агент.
   `POST /api/tickets/{id}/analyze`

3. **Агент читает факты**
   Обращение, клиент без email, история, лицензия по ключу оператора - только чтение.
   `get_ticket · get_license`

4. **Агент ищет статьи**
   Статьи продукта обращения и общие статьи в Qdrant.
   `search_knowledge`

5. **Черновик со ссылками**
   summary, category, draft_reply, sources. Запуск completed, предложение pending.
   `GET /api/agent-runs/{id}`

6. **Граф ждёт человека**
   interrupt(): состояние сохранено в PostgreSQL, ждать можно сколько угодно.
   `GET …/workflow → awaiting_review`

7. **Сотрудник решает**
   approve - можно отредактировать текст; reject - с причиной.
   `POST /api/suggestions/{id}/review`

8. **Ответ доставлен**
   Сообщение оператора в обращении, статус waiting_customer, граф delivered.
   `ticket_messages`

Очередь на проверку: `GET /api/suggestions?status=pending`. Каждый повторный анализ создаёт новый запуск и новое предложение; решение по предложению принимается один раз.

4 Как работает агент

Модель просит факты, сервер решает, что можно выполнить, и проверяет итог.



Инструмент	Что возвращает	Ограничение
`get_ticket`	Текущее обращение и продукт	только обращение этого запуска
`get_customer`	Имя и компания клиента	email не выдаётся
`get_license`	Статус и признак valid по датам	только клиент и продукт обращения; ключ в журнале - ***
`get_ticket_history`	До 5 других обращений	тот же клиент и продукт, без описаний
`search_knowledge`	До 5 статей, по фрагменту от каждой	продукт задаёт сервер: свои статьи + общие

⩽ 6	⩽ 12	⩽ 60 с	64 000
шагов модели за запуск	вызовов инструментов	на весь цикл	символов истории максимум

Безопасность: имя функции и JSON модели никогда не превращаются в код или SQL. Транзакции инструментов - **READ ONLY** на уровне PostgreSQL. Ссылка в `sources` на статью, которую поиск не находил, останавливает запуск с ошибкой 502. Ошибка после старта сохраняется как **failed** с `run_id`, без предложения.

5 База знаний и поиск (RAG)

RAG - ответ с опорой на найденные документы. Сначала поиск, потом черновик со ссылками.



Повторный запуск: без изменений → unchanged;
изменённая статья → переиндексирована;
удалённая из каталога → убрана отовсюду.

Как работает поиск

Текст запроса превращается в вектор той же моделью, что и статьи. Qdrant сравнивает его с векторами фрагментов (косинусное сходство: 1 - одинаковое направление векторов, около 0 - слабая связь векторов) и отбрасывает всё ниже порога. С кодом продукта ищутся статьи этого продукта и общие; агенту продукт подставляет сервер.

Запрос: «Где найти журнал сервера и собрать диагностику»



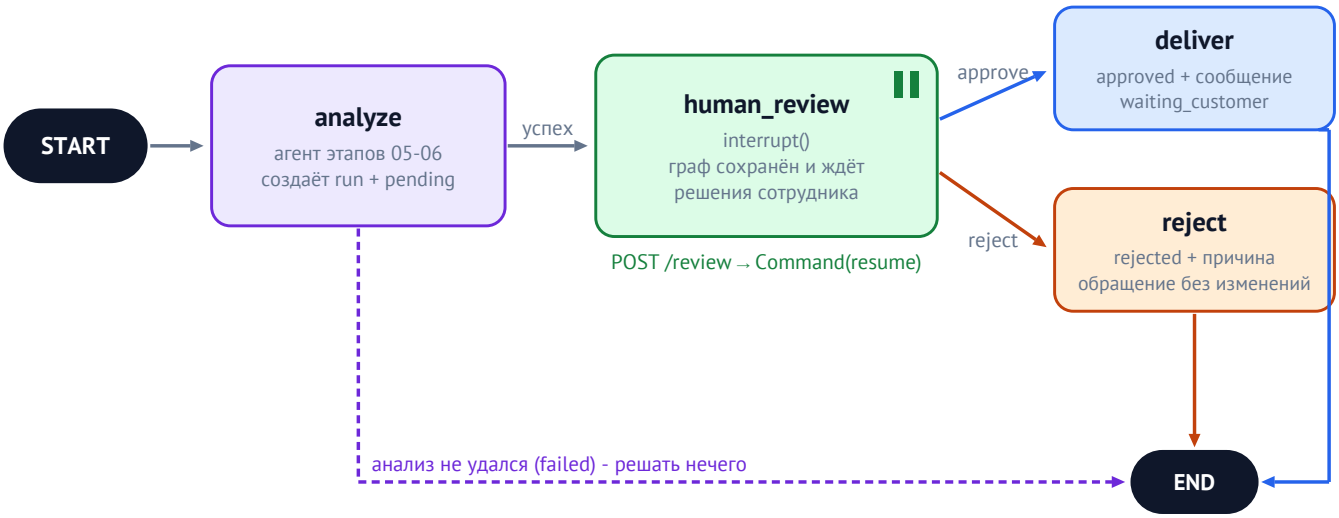
Серые столбцы ниже порога в выдачу не попадают. Данные - настоящий поиск mock-эмбеддингов по 27 фрагментам без фильтра продукта.

Mock-эмбеддинг - лексический: слова приводятся к основе из 5 букв («ошибка», «ошибке» → «ошибк»), каждая основа через SHA-256 попадает в одну из 256 координат. Он находит тексты с общими словами, но не понимает синонимы. Адаптер OpenAI-эмбеддингов готов; после смены модели статьи нужно переиндексировать.

6 LangGraph и решение человека

Процесс как граф: после анализа он останавливается и ждёт сотрудника.

После каждого узла LangGraph сохраняет состояние потока (checkpoint) в PostgreSQL.



Решение	Что передать	Что произойдёт
approve	reviewer_id; можно final_content - исправленный текст	Предложение approved, сообщение оператора в обращении, статус waiting_customer, граф delivered
reject	reviewer_id и обязательный comment с причиной	Предложение rejected, обращение без изменений, граф rejected

Проверка до продолжения

Неверный сотрудник получает 404, а граф продолжает ждать: ожидание не тратится на ошибку.

Одно решение

Advisory lock и SELECT ... FOR UPDATE: из двух одновременных решений применяется одно, второе получает 409.

Перезапуск не страшен

Состояние в PostgreSQL. CI пересоздаёт контейнеры и продолжает отложенный граф.

Без лишних данных

В checkpoint только id и итоги; ключ лицензии туда не попадает.

Неудачный анализ

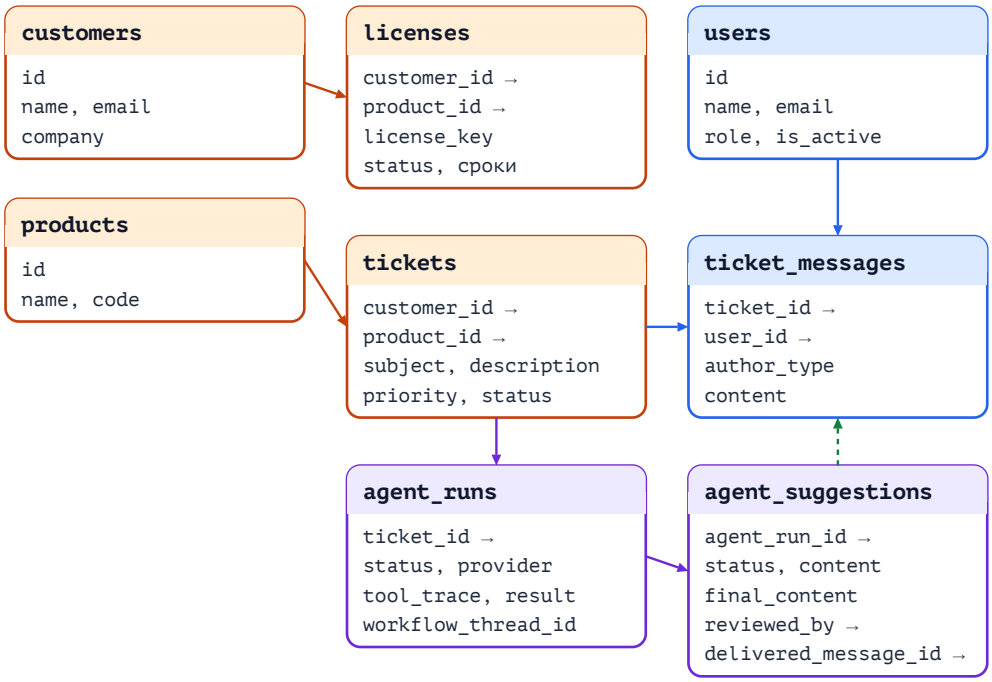
Запуск failed завершает граф: состояние failed, предложения нет.

Это не авторизация

reviewer_id - учебная отметка. Вход и права сотрудников - будущая работа.

7 Данные: таблицы и миграции

Что хранится в PostgreSQL и как схема менялась от этапа к этапу.



Состояние графов LangGraph (миграция 0004). Без моделей SQLAlchemy - alembic check их пропускает.

Тексты и векторы статей - в Qdrant; здесь только реестр.

Стрелки - внешние ключи; пунктир - сообщение, созданное доставкой одобренного ответа.

Миграции Alembic



Схему меняет **только Alembic**: сервис migrate применяет миграции до старта API, а `/ready` отвечает 200, лишь когда версия БД совпадает с кодом. Старая миграция не переписывается - каждое изменение получает новую версию. Проверки отката выполняются только на одноразовой тестовой БД.

8 Шпаргалка API

22 URL-пути и 27 HTTP-операций. Интерактивная документация - http://127.0.0.1:8001/docs.

Раздел	Метод	Путь	Что делает
Состояние	`GET`	`/health`	Процесс API жив
	`GET`	`/ready`	PostgreSQL доступна, миграции актуальны
Ресурсы	`POST` / `GET`	`/api/customers`	Создать или список клиентов
	`POST` / `GET`	`/api/products`	Продукты
	`POST` / `GET`	`/api/licenses`	Лицензии (фильтры клиента, продукта, ключа)
	`POST` / `GET`	`/api/tickets`	Обращения (новые всегда open)
	`POST` / `GET`	`/api/users`	Сотрудники поддержки
	`GET`	`/api/{ресурс}/{id}`	Одна запись любого ресурса выше
LLM	`GET`	`/api/llm/status`	Какой провайдер настроен
	`POST`	`/api/tickets/{id}/draft`	Предварительный черновик без сохранения
База знаний	`GET`	`/api/knowledge/status`	Индекс, точки, статьи реестра
	`POST`	`/api/knowledge/search`	Поиск фрагментов, фильтр product_code
Агент	`POST`	`/api/tickets/{id}/analyze`	Анализ; запускает граф проверки
	`GET`	`/api/agent-runs/{id}`	Сохранённый запуск: trace, итог, предложение
Проверка	`GET`	`/api/agent-runs/{id}/workflow`	Состояние графа LangGraph
	`GET`	`/api/suggestions`	Очередь, фильтры status и ticket_id
	`GET`	`/api/suggestions/{id}`	Предложение и решение по нему
	`POST`	`/api/suggestions/{id}/review`	Одобрить или отклонить

Коды ответов

Код	Значение	Код	Значение
200 / 201	успех / запись создана	**413**	контекст для модели слишком большой
404	запись не найдена	**502**	непригодный ответ модели или чужая ссылка
409	конфликт: дубликат, уже рассмотрено	**503**	БД, Qdrant или провайдер недоступны
422	неверный запрос: поля, типы, лишнее	**504**	истёк тайм-аут модели или поиска

9 Как запустить у себя

Нужны Python 3.12+, uv и Docker Desktop. Команды - из корня проекта.

1 Настройки и зависимости

```
cp -n .env.example .env
uv sync --locked
```

cp -n не перезапишет существующий .env.

2 Запустить стенд

```
docker compose up --build --detach --wait api
```

Поднимет db, qdrant, применит миграции и запустит API.

3 Проиндексировать статьи

```
docker compose exec -T api python -m app.knowledge.ingest
```

Повторный запуск ничего не перепишет.

4 Проверить всё одной командой

```
uv run --locked python scripts/smoke_test.py \
  --base-url http://127.0.0.1:8001 --timeout 60 --check-review
```

Создаёт учебные записи: не запускай на рабочей базе.

5 Открыть Swagger

```
open http://127.0.0.1:8001/docs
```

Там можно вызвать любой маршрут из браузера.

Адрес	Что там
`http://127.0.0.1:8001/docs`	Swagger UI - все операции
`http://127.0.0.1:8001/ready`	{"status":"ok","database":"ok"} - БД и миграции готовы
`127.0.0.1:5433 / 127.0.0.1:6335`	PostgreSQL / Qdrant для инструментов на Mac

Остановить: `docker compose down` - данные сохранятся. Чтобы подключить OpenAI, задай в .env `LLM_PROVIDER=openai`, ключ и модель; это платно и отправляет данные провайдеру.

10 Проверки качества и CI/CD

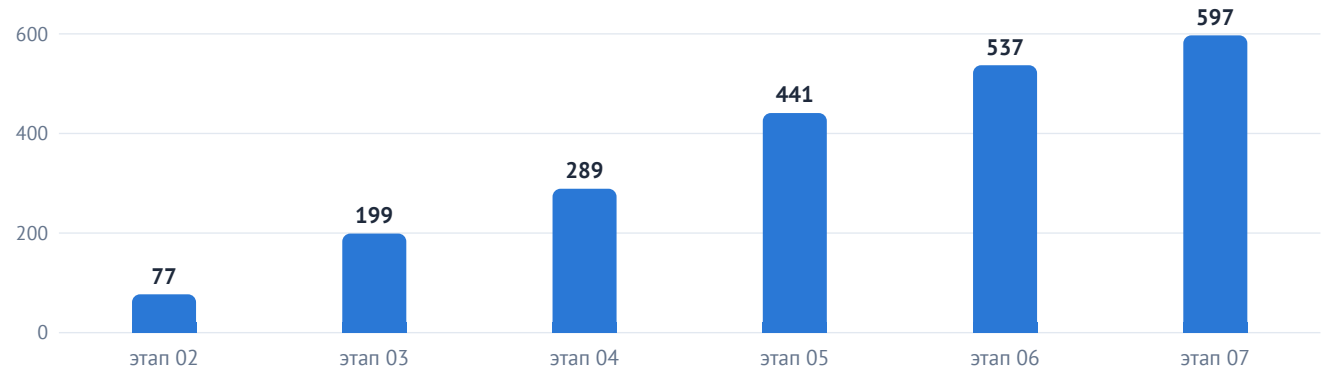
Каждое изменение проверяется автоматически, прежде чем образ попадёт в реестр.

522
unit-тестов: без сети, БД - заглушки и Qdrant в памяти

113
интеграционных: настоящие PostgreSQL и Qdrant

89
включены в unit: тесты smoke CLI; сам smoke - на живом стенде

Сколько тестов было после каждого этапа



Конвейер GitHub Actions

1. checks + security
lock, Ruff и документы миграции + Pytest
actionlint и pip-audit
PostgreSQL + Qdrant

успех →

2. Docker + smoke
сборка образа, Compose
индексация статей
--check-review
пересоздание контейнеров: данные, индекс, граф живы

успех →

3. Публикация в GHCR
только push в main
тот же проверенный образ
тег sha-<коммит>
и latest

Тесты никогда не трогают рабочую БД: интеграционные требуют явный адрес базы с суффиксом `_test`, Qdrant - коллекции `test_*`. Unit-графы используют память; интеграционные review - PostgresSaver в отдельной тестовой БД. Итог удалённых запусков смотри в GitHub Actions: локальная проверка его не заменяет.

11 Дорожная карта

Десять этапов учебного проекта: семь уже готовы.

01	**FastAPI, /health, первые тесты** основа API, uv, Ruff, Pytest	готово
02	**PostgreSQL, модели, миграции** 9 таблиц, Alembic, /ready, Docker Compose	готово
03	**REST API ресурсов** клиенты, продукты, лицензии, обращения	готово
04	**LLM-черновик, mock и OpenAI** строгий JSON, безопасные ошибки	готово
05	**Агент с инструментами** цикл tool calling, лимиты, сохранённый запуск	готово
06	**База знаний, RAG, Qdrant** индексация статей, поиск, ссылки агента	готово
07	**LangGraph и решение человека** одобрение, доставка, checkpoint в БД	готово
08	**Интерфейс оператора** Streamlit поверх готового API	в планах
09	**Оценки качества и безопасность** наборы вопросов, проверки агента	в планах
10	**Упаковка и демонстрация** материалы для показа на GitHub	в планах

Каждый этап - отдельная пошаговая инструкция на русском: `docs/STAGE_01.md` … `STAGE_07.md`. В журнале `docs/DEVELOPMENT_LOG.md` записано, что именно проверено и чем.

12 Словарь и куда читать дальше

Термины, которые встречаются в проекте.

Термин	Простыми словами
API	Набор адресов (маршрутов), через которые программы общаются с сервером.
FastAPI	Фреймворк Python для HTTP API; сам строит документацию /docs.
Миграция	Версия изменения схемы БД; Alembic применяет их по порядку.
Docker Compose	Запуск нескольких связанных контейнеров одной командой.
LLM / mock	Языковая модель / локальная заглушка без сети и платы.
Агент	Программа, которая просит инструменты, получает факты и готовит итог.
Tool calling	Протокол: модель просит вызвать функцию, сервер её выполняет.
RAG	Ответ с опорой на найденные документы и ссылками на них.
Эмбеддинг	Список чисел, описывающий текст; близкие тексты - близкие векторы.
Qdrant	Векторная БД: быстро находит похожие векторы с фильтрами.
LangGraph	Библиотека для процессов-графов с состоянием и остановками.
Checkpoint	Сохранённое состояние графа после шага; здесь - в PostgreSQL.
Interrupt / resume	Остановка графа до решения и продолжение с этим решением.
Human in the loop	Шаг, где решение принимает человек, а не модель.
Smoke-тест	Быстрая проверка, что запущенная система отвечает как надо.
CI/CD, GHCR	Автопроверки на GitHub и публикация Docker-образа в реестр.

Куда читать дальше

Файл	Что там
`README.md`	Краткое описание, дорожная карта, оглавление
`docs/instructions/`	Быстрый запуск, работа с API, тесты, CI, структура проекта
`docs/STAGE_01-07.md`	Пошаговые учебные инструкции по этапам на русском
`docs/APPROACH.md`	Архитектура и решения (англ.)
`docs/CI_CD.md`	Подробно о GitHub Actions и GHCR
`docs/DEVELOPMENT_LOG.md`	Что сделано и как проверено на каждом этапе

13 Итоги полной проверки

Результаты относятся к этой версии исходников; удалённый CI проверяется отдельно.

Дата аудита: **2026-10-18**. Исходный коммит: `f6e72ae`. Проверяются приложение, миграции, поиск, процесс решения, образ и документация.

Группа	Результат
Полный Pytest	635 passed; 0 skipped
Unit / интеграционные	522 / 113
Smoke CLI (входит в unit)	89
Контракт HTTP	22 URL-пути, 27 операций

Проверка	Фактическое свидетельство
Код и комментарии	Ruff: 111 файлов; 843 функции/класса с docstrings.
Инструкции	Локальные ссылки, Bash без выполнения и JSON проверены.
Workflow	actionlint 1.7.12; 27 shell-блоков проверены bash -n.
Зависимости	69 runtime-пакетов; 0 известных уязвимостей, 0 пропусков.
Миграции	0004_review_workflow; alembic check без изменений.
Docker и HTTP	UID/GID 10001; без devtools; полный mock review smoke.
Сохранность	Данные, поиск и checkpoint сохранены; resume после пересоздания.
Проверенный образ	save/load сохраняет image ID; GHCR настроен без пересборки.

> Удалённый запуск итогового коммита GitHub Actions и его публикация GHCR ещё не подтверждены. Локальные проверки выполнены успешно.

Как читать эту страницу

Число тестов показывает объём автоматических сценариев. Оно не измеряет качество ответов настоящей модели. Для GitHub нужен результат запуска именно итогового коммита; локальный успех и наличие YAML не подтверждают публикацию нового образа.

14 CI: матрица проверок

Каждый push в main проходит последовательные барьеры перед публикацией образа.

Событие	Проверки / образ	GHCR
Локальный git commit	Не запускаются на GitHub	Нет
Push в main	Все обязательные задания	После успеха
PR с целью main	checks + security → container	Нет
Run workflow вручную	checks + security → container	Нет

Барьер	Что проверяется	Зачем
Зависимости	uv sync --locked; pip-audit 2.10.1 по хешам	Отдельный security job; известные уязвимости
checks: код и документы	Ruff, формат, check_project.py	Ошибки кода, комментарии, ссылки
Workflow	YAML, выражения и задания через actionlint	Ошибки задания до запуска на GitHub
PostgreSQL + Qdrant	Миграции, alembic check, весь Pytest	Реальная схема, данные и поиск
Контейнер	Сборка, readiness, UID и runtime	Запускается готовый образ без root
Полный сценарий	Ingest; агент; review; повтор решения	Путь от обращения до сообщения
Пересоздание	Данные, индекс, pending workflow	Состояние сохраняется в томах
Доставка	Загрузить тот же образ; сверить ID	Публикуется проверенная сборка

Ошибка обязательного задания останавливает цепочку. Следующие задания могут стать Skipped: сначала ищи первый неуспешный шаг. На PR пропуск publish ожидаем. Полный состав шагов определяет .github/workflows/ci.yml.

Один образ на весь путь

container собирает образ и проверяет его. В publish передаются архив и сохранённый image ID; после загрузки ID сравнивается. Повторной сборки перед GHCR нет. Это защищает от ситуации, когда тестируется одна сборка, а доставляется другая.

15 Защита и доставка

Технические ограничения проекта, права GitHub и воспроизводимая версия образа.

Область	Как устроено
Права GitHub	contents: read по умолчанию; packages: write только в publish
Внешние Actions	Версии закреплены полным SHA; actionlint 1.7.12 игнорирует только новое queue: max
Секреты	GITHUB_TOKEN выдаётся заданию; .env и ключи не входят в образ
Режим CI	LLM и embeddings = mock; OpenAI credentials пустые
Процесс API	Постоянный пользователь UID 10001; dev-инструментов в runtime нет
Агент	Строгие аргументы, серверная область данных, READ ONLY инструменты
Тесты	Отдельная *_test БД, временные test_* коллекции Qdrant
Публикация	Только успешный push в main; SHA-тег для каждого коммита
Тег latest	Обновляется, только если публикуемый SHA всё ещё вершина main

Как получить ровно проверенную версию

Actions → CI and GHCR → нужный коммит → Publish tested image to GHCR. В Summary скопируй полный тег sha-... или digest. latest меняется со следующими версиями; digest обозначает содержимое конкретного образа.

```
read -r -p "Полный SHA из Summary: " IMAGE_SHA
export APP_IMAGE="ghcr.io/merkertg/ai-support-agent:sha-$IMAGE_SHA"
docker pull "$APP_IMAGE"
docker compose up --detach --no-build --wait api
```

> Для закрытого GHCR-пакета требуется вход аккаунтом с доступом. Локальный токен read:packages передавай через скрытый ввод/--password-stdin. Не добавляй его в .env.example, код или историю команд.

Граница реализованной защиты

Аудит зависимостей ищет известные уязвимости на дату проверки. Проверки не заменяют авторизацию и оценку качества ответов. reviewer_id остаётся учебной отметкой; входа сотрудников и прав доступа ещё нет. CD заканчивается в GHCR: внешняя площадка развёртывания пока не выбрана.

16 Полный локальный прогон

Команды из корня проекта. Для сброса схемы используются отдельные тестовые сервисы.

1. Подготовить окружение и одноразовые зависимости

```
cp -n .env.example .env
uv sync --locked
docker compose --profile test up --detach --wait \
  db-test qdrant-test
```

2. Проверить код и документацию

```
uv run --locked ruff check .
uv run --locked ruff format --check .
uv run --locked python scripts/check_project.py
```

3. Выполнить весь Pytest без пропусков

```
export TEST_DATABASE_URL='postgresql+psycopg://support:'\
'support_ci@127.0.0.1:5434/ai_support_test'
export TEST_QDRANT_URL=http://127.0.0.1:6336
CI=true uv run --locked pytest -v
unset TEST_DATABASE_URL TEST_QDRANT_URL
```

4. Проверить собранный стенд через HTTP

```
docker compose up --build --detach --wait api
docker compose exec -T api python -m app.knowledge.ingest
docker compose exec -T api python -m alembic check
uv run --locked python scripts/smoke_test.py \
  --base-url http://127.0.0.1:8001 \
  --timeout 60 --check-review
```

Ожидается код завершения 0 у всех команд, ноль пропусков в полном Pytest, миграции без различий и успешные шаги smoke. --check-review создаёт учебные записи: выполняй на учебном стенде. Без TEST URLs локальные интеграционные тесты пропускаются; CI=true превращает отсутствие адресов в ошибку.

17 Практика оператора

Полный путь через Swagger: можно повторить без программирования и ключа модели.

Открой http://127.0.0.1:8001/docs. В нужной операции нажми Try it out, заполни JSON и Execute. ID бери из ответов: они различаются на разных стендах.

Шаг и операция	Что передать / что увидеть
1. GET /api/llm/status	provider=mock, configured=true; затем GET /api/knowledge/status: index=ready
2. POST /api/customers	name: Учебный клиент; email: новый адрес @example.com. Сохранить customer.id
3. Продукт и лицензия	GET /api/products: взять techflow-server. Если его нет, POST /api/products: name=TechFlow Server, code=techflow-server. Затем POST /api/licenses: customer_id, product_id, новый license_key
4. POST /api/users	name и новый email сотрудника. Сохранить reviewer_id; role по умолчанию operator
5. POST /api/tickets	customer_id, product_id, subject и description проблемы. Начальный статус open
6. POST .../{id}/analyze	{"license_key":"ключ шага 3"}. Запуск completed, предложение pending, sources из найденных статей
7. GET .../{run_id}/workflow	state=awaiting_review, next=[human_review]. Это ожидание сотрудника
8. POST /api/suggestions/{id}/review	decision=approve, reviewer_id; можно final_content с исправленным ответом
9. Проверить результат	suggestion approved, message operator, ticket_status waiting_customer, workflow delivered
10. Повторить решение	HTTP 409: предложение уже рассмотрено, второго сообщения нет

Отдельно пройти отклонение

Создай второе обращение и запусти анализ. Для pending предложения передай decision=reject, reviewer_id и обязательный comment с причиной; final_content не передавай. Ожидаются rejected, message=null, обращение остаётся open. Пошаговые curl-блоки с сохранением ID находятся в docs/STAGE_07.md, разделы 9-10.

> Перед одобрением прочитай источники и исправь ответ. Mock показывает путь данных, а не качество технической рекомендации. «Доставка» здесь означает сообщение в PostgreSQL, не письмо и не ответ во внешней системе.

18 Перезапуск и данные

Остановка контейнеров, сохранность томов и продолжение решения сотрудника.

Состояние	Где хранится	После обычного down / up
Предметные записи и run	Том postgres_data	Сохраняются
Решение и сообщение	PostgreSQL	Сохраняются
Ожидание human_review	checkpoint_* в PostgreSQL	Можно продолжить
Векторы статей	Том qdrant_data	Сохраняются
Учебные db-test/qdrant-test	tmpfs / временные данные	Одноразовые

Сравнить предметные данные до и после

```
docker compose exec -T api python -m app.db.demo seed \
  > /tmp/ai-support-before.json
docker compose down
docker compose up --detach --no-build --wait api
docker compose exec -T api python -m app.db.demo show \
  > /tmp/ai-support-after.json
cmp /tmp/ai-support-before.json /tmp/ai-support-after.json
```

cmp с кодом 0 без вывода подтверждает совпадение снимков. После up не выполняй повторный seed до сравнения: он мог бы заново создать потерянные записи.

Проверить ожидающий процесс

Создай анализ и сохрани run_id, оставив предложение pending. Пересоздай контейнеры обычным down/up. GET /api/agent-runs/{id}/workflow должен снова показать awaiting_review. Прими решение через review: граф продолжится из checkpoint. CI повторяет этот сценарий автоматически и дополнительно сравнивает индекс поиска. При сбое saver чтение workflow восстанавливает checkpoint из AgentRun/AgentSuggestion под advisory lock. Анализ и доставка при восстановлении не повторяются.

> Обычный docker compose down сохраняет именованные тома. Параметр --volumes удаляет данные и применяется CI только к своему изолированному проекту. Сохранение тома не заменяет резервное копирование. Автоматический backup и восстановление аварийно зависших running пока не реализованы.

19 Диагностика ошибок

Начинай с первого неуспешного шага; повторная публикация не исправляет ошибку тестов.

Симптом	Что проверить	Следующее действие
Docker daemon недоступен	Docker Desktop и docker version	Запустить Docker, дождаться готовности
Порт занят	compose ps; API_PORT, POSTGRES_PORT	Выбрать свободный порт и согласовать URL
/health 200, /ready 503	Логи db/migrate; Alembic current	Исправить БД или применить миграцию
Индекс missing	GET /api/knowledge/status	Выполнить явный ingest
Индекс incompatible	Модель и размерность embeddings	Согласовать настройки, переиндексировать
HTTP 422	JSON, лишние поля, ID, поля решения	Сверить схему Swagger; reject требует comment
HTTP 404 reviewer	Сотрудник существует и активен	Выбрать действующий reviewer_id
HTTP 409 review	Состояние предложения	Прочитать уже принятое решение
HTTP 502 / 504	run_id, safe error, лимиты	Изучить failed run, исправить причину
Тесты skipped локально	TEST_DATABASE_URL, TEST_QDRANT_URL	Поднять test-сервисы, повторить полностью
GitHub: container Skipped	Результат предыдущего задания	Исправить первый сбой checks/security
GHCR denied	Права пакета, владелец, GITHUB_TOKEN	Проверить Packages/Actions permissions

Команды для сбора понятной диагностики

```
docker compose ps --all
docker compose logs --no-color --tail 100 \
  api migrate db qdrant
docker compose exec -T api python -m alembic current
```

В GitHub открой CI and GHCR → нужный коммит → первый красный шаг. JUnit лежит в pytest-report; контейнерные логи показывают зависимости. После исправления повтори весь workflow, если архив проверенного образа уже удалён. Новый Run workflow выполняет проверки без publish.

20 Что делать дальше

Готовый API и CI дают основу для интерфейса; реальную модель подключают отдельно.

Что изменено при этом аудите

• Исправлено восстановление checkpoint после сохранённого анализа или решения без повторной доставки.
• Ошибки psycopg и тайм-аут пула возвращают безопасный HTTP 503; чужие id checkpoint отклоняются.
• Код general зарезервирован для общих статей; продуктовая статья не попадает в общий поиск.
• Добавлено 38 автоматических сценариев; CI дополнен аудитом зависимостей, проверками workflow, инструкций и runtime-образа.
• Обновлены инструкции и воспроизводимый исходник этого PDF; сохранена резервная копия прежнего документа.

Следующая работа	Критерий готовности
Этап 08: Streamlit	Список и карточка обращений; анализ; источники; pending очередь; approve/reject через API
Этап 09: оценки	Размеченные вопросы, ожидаемые статьи, метрики поиска и критерии ответа; негативные сценарии
Реальный OpenAI	Пользователь задаёт ключ и доступную модель локально; отдельный ограниченный ручной запрос
Вход и роли	Сервер проверяет личность и право принимать решение; reviewer_id из доверенного контекста
Этап 10: показ	Воспроизводимый сценарий, скриншоты, примеры API и проверяемые результаты
Внешний сервер	Выбрана площадка, настроены секреты/backup; проверка работающего сервиса после deploy

Остающиеся ограничения

• Реальные OpenAI-запросы и качество ответов не проверялись; выбран mock без ключа.
• Интерфейс Streamlit, авторизация и оценки качества остаются следующими этапами.
• Учебная доставка сохраняет сообщение в PostgreSQL; внешняя отправка и серверный deploy ещё не настроены.

> OpenAI-адаптеры уже есть, но реальный запрос и качество ответов не проверены. По текущему выбору пользователя сохраняется mock: ключ не нужен и платных вызовов нет. Подключение настоящей модели меняет стоимость и передачу данных; оно выполняется отдельным явным действием.

Документация и проверяемая история

Репозиторий проекта · GitHub Actions · Документация

Исходник этого PDF: scripts/build_guide.py. Проверенные числа и результаты: docs/guide/verification.json. Инструкция пересборки: docs/guide/README.md. Не меняй числа вручную в макете: обновляй свидетельства после полного прогона.